

Driver JDBC tradicional y bloqueante en el backend

Conclusión

Sí, el backend utiliza un driver JDBC tradicional y bloqueante.

Todo el acceso a datos es síncrono y bloqueante mediante JDBC + Hibernate (JPA). No hay ningún componente reactivo (ni R2DBC, ni WebFlux, ni drivers no bloqueantes) en el proyecto.

Evidencia en `backend/pom.xml` (dependencias)

Dependencia	Rol
<code>spring-boot-starter-data-jpa</code>	Hibernate sobre JDBC síncrono.
<code>com.oracle.database.jdbc:ojdbc11</code>	Driver JDBC clásico de Oracle (bloqueante).
<code>spring-boot-starter-web</code>	Modelo servlet (Tomcat), no WebFlux.

Además, **no** hay ninguna dependencia de R2DBC ni de drivers reactivos en el archivo `pom.xml`.

Evidencia en `backend/src/main/resources/application.properties`

```
# Datasource Oracle
spring.datasource.url=jdbc:oracle:thin:@//localhost:1521/XEPDB1
spring.datasource.username=TASKFLOW_USER
spring.datasource.password=TaskFlowI23
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.OracleDialect
spring.jpa.show-sql=false
spring.jpa.open-in-view=false
```

Puntos clave:

- `spring.datasource.url=jdbc:oracle:thin:@//localhost:1521/XEPDB1` → URL JDBC tradicional.
 - `spring.datasource.driver-class-name=oracle.jdbc.OracleDriver` → driver JDBC clásico.
 - `spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.OracleDialect` → Hibernate con dialecto Oracle.
-

Implicación

Cada petición HTTP que toca la base de datos ocupa un hilo del contenedor de servlets (Tomcat) mientras espera la respuesta de Oracle. Esto significa que:

- El modelo de concurrencia es **1 hilo por petición** (thread-per-request).
 - Bajo alta concurrencia, el pool de hilos de Tomcat puede agotarse si las consultas a Oracle son lentas.
 - No hay beneficios de programación reactiva (no bloqueante) en el acceso a datos.
-

¿Cómo sería un enfoque reactivo (alternativa)?

Para hacer el acceso a datos no bloqueante se necesitaría:

- `spring-boot-starter-webflux` en lugar de `spring-boot-starter-web`.
- Un driver reactivo como `r2dbc-oracle` en lugar de `ojdbc11`.
- Repositorios reactivos (`ReactiveCrudRepository`) en lugar de `JpaRepository`.
- Hibernate Reactive en lugar de Hibernate clásico.

Actualmente el proyecto **no** implementa ninguna de estas alternativas.